PARTICULA PRIMORDIAL

An Auditable Toolkit for MIDI Normalization, Transposable Scale Studies and Computational Composition

Technical note | Version 0.2.0 | 1 November 2026 | Open-source research software

Abstract

Abstract. Particula Primordial is an open-source prototype for a reproducible workflow that connects MIDI event analysis, temporal normalization, transposable tonal material, algorithmic composition and digital-audio-workstation project inspection. Its primary input is a user-supplied Standard MIDI File. The system reconstructs note events, represents positions in quarter-note beats, and can standardize onset timing to a configurable grid, note durations to a fixed value, tempo to a single target BPM and meter to 4/4. The normalization stage preserves pitch, velocity and channel for selected note streams and does not perform automatic pitch correction. A parameterized scale engine represents each scale as an interval vector relative to a tonic, allowing the same recipe to be transposed across all twelve pitch classes in 12-TET. A composition engine derives extended sonorities, applies seven progression templates and generates deterministic melodic contours. A conservative REAPER project parser and a host-side ReaScript exporter provide an optional DAW workflow. Automated tests verify selected software invariants using synthetic fixtures. The prototype does not claim transcription accuracy, perceptual superiority or production-grade synthesis; those properties require separate empirical evaluation.

Keywords: computational musicology; MIDI; quantization; music information retrieval; algorithmic composition; scale transposition; REAPER; Streamlit; reproducibility.

1. Introduction

A short piano scale recording or MIDI exercise is useful source material for further study, but its event timing may be inconsistent. Note onsets can fall between intended beat locations, note lengths can vary unintentionally, and file metadata can complicate notation-oriented work. A practical workflow should preserve performed pitch content while making temporal organization explicit and reproducible.

Particula Primordial addresses this problem and extends it into a compositional research environment. The source is not assumed to be in a particular key. A user chooses a tonic and an interval recipe; the same procedure can then generate scale exercises and composition studies in another key without rewriting the algorithm.

The platform uses Standard MIDI Files as the canonical note-event interchange format. MIDI timing is tick-based: events store delta times, and PPQ resolution together with tempo metadata determines the mapping between ticks, beats and seconds [1]. The software therefore distinguishes beat-domain quantization from second-domain audio measurements.

2. System architecture

The application consists of five interoperable layers: (1) input and parsing of MIDI, WAV and REAPER project text; (2) note-event reconstruction; (3) temporal normalization; (4) tonal and compositional generation; and (5) visualization and export. The Streamlit dashboard presents the workflow through an upload-oriented interface.

Input	Processing	Output
MIDI / WAV / RPP	Parsing, event reconstruction, audio features, project inventory	Auditable tables and project metrics
Tonic + interval recipe	Transposition, tertian stacking, algorithmic contour generation	Scale, progression and ascension MIDI
MIDI composition	Reference synthesis or local REAPER rendering	WAV / optional MP3 / production render

The architecture exposes two execution modes. Offline mode operates on uploaded files and does not require REAPER. The host-dependent mode delegates full arrangement playback and rendering to REAPER, which owns the project's media, plugins, routing and render settings. REAPER project files (`.RPP`) are text-based and hold track, media and settings information [2]. ReaScript exposes MIDI note and event functions [3], and REAPER's command-line interface provides a `-renderproject` option [4].

The RPP reader in this repository is limited to project inventory: tempo and meter when present, track names, media-item positions and lengths, source references and visible effect-chunk labels. Media-path accessibility is evaluated only in the process environment and cannot prove availability on the original user workstation. It is not a complete parser of every REAPER chunk or plugin state. A separate Lua ReaScript uses the host API to export note-level information from selected MIDI takes as CSV.

3. MIDI event model and temporal normalization

Within a MIDI track, delta times are accumulated into absolute tick positions. A positive-velocity note-on starts a note; a matching note-off or zero-velocity note-on closes it. Matching uses MIDI channel and pitch. Each reconstructed event is represented by pitch, velocity, channel, onset tick and offset tick. With PPQ resolution P, beat position is t/P. The normalized file retains the source PPQ.

For a user-selected onset grid g in quarter-note beats, the quantized onset is:

```
q_i = round((b_i / g)) * g
```

The earliest quantized onset is shifted to beat zero: q_i_new = q_i - min(q). With uniform duration enabled, the output interval becomes [q_i_new, q_i_new + d], where d is the selected duration in beats. Otherwise, the source duration is rounded to the selected grid with a minimum of one grid interval. The output receives one target BPM and a 4/4 time-signature event.

4. Transposable scale representation

A scale is represented as an ascending semitone interval vector S = (0, s1, s2, ..., sm). For tonic pitch class r, each generated pitch class is (r + si) mod 12. This makes the scale engine independent of key. The starter catalogue contains 21 practical recipes: major, natural minor, harmonic minor, ascending melodic minor, modal, pentatonic, blues, whole-tone, diminished and selected synthetic collections, plus chromatic. It is a practical 12-TET set, not an exhaustive taxonomy of scales across musical cultures, tuning systems or microtonal practice.

A generated scale MIDI contains a track name, tempo and 4/4 signature. The exercise ascends through the selected octave range and returns in descending order. A manifest records tonic, recipe name, interval vector and filename for reproducibility.

5. Scale-derived harmony and algorithmic melody

The composition engine uses seven four-chord degree templates. For each chord root, alternate scale degrees are stacked to create triadic and extended sonorities. In a seven-note collection, a 13th stack uses degrees 1, 3, 5, 7, 9, 11 and 13. For non-heptatonic collections, the stack cycles through available scale degrees; conventional chord-symbol labels may be approximate, while the MIDI pitches remain traceable to the interval recipe.

Study	Degree pattern (1-based)	Melody contour
01	1 - 6 - 4 - 7	Mirror / reflection
02	1 - 4 - 7 - 3	Modular permutation
03	6 - 4 - 5 - 1	Fibonacci residues
04	4 - 1 - 6 - 7	Prime residues
05	1 - 7 - 6 - 3	Logistic map
06	2 - 1 - 4 - 6	Bounded random walk
07	4 - 6 - 1 - 7	Cyclic pendulum

The seven melodic methods generate deterministic scale-degree indices from an algorithm number and seed. These are controlled compositional transformations, not claims that a mathematical sequence is inherently musical. Musical evaluation must consider voice-leading, register, rhythmic hierarchy, repetition and listening perception.

The ascension study sustains a tonic-centered pedal field while an upper voice moves through the selected collection. Its length is configurable. The final sonority remains open rather than providing a conventional cadence. It is extensible but finite in any exported MIDI file.

6. Audio measurement and rendering

The WAV feature module reports duration, sample rate, channel count, absolute peak, RMS, zero-crossing rate and a frame-averaged spectral-centroid estimate. These measurements describe selected signal properties; they do not by themselves determine perceived loudness, timbre quality, musicality or production quality.

A minimal additive renderer creates reference WAV previews from MIDI notes. MP3 conversion is performed through FFmpeg when installed. This renderer is not represented as a piano sample library or a replacement for REAPER. For DAW-native audio, REAPER must resolve the project's own instruments, effects, routing and render configuration. Streamlit Community Cloud supports deployment from a GitHub repository with dependency files [5], but the hosted application cannot assume access to a user's local DAW or proprietary plugins. FluidSynth provides an alternative SoundFont-based MIDI rendering route when a compatible SoundFont and runtime are installed [6].

7. Dashboard and reproducibility

The Streamlit dashboard groups the workflow into overview, MIDI normalization, scale catalogue, composition lab, REAPER/audio analysis and research notes. Output artifacts are downloadable individually or as ZIP archives. CSV manifests and JSON project summaries provide traceable records of selected parameters.

Reproducibility depends on recording the source file, software version, PPQ, target BPM, grid, duration, tonic, scale interval vector, progression template, algorithm and seed. A future version should add content hashes and a formal experiment log for every export.

8. Validation strategy

The automated test suite uses synthetic fixtures to test invariants: tonic parsing and enharmonic equivalence; scale transposition; chord pitch membership; note pitch and velocity preservation during normalization; onset-grid alignment; fixed output duration; 4/4 and tempo metadata; composition track structure; reference WAV validity; and expected RPP inventory extraction.

These tests validate implementation behavior, not artistic outcomes. Empirical evaluation remains necessary. A future study should use a corpus of scale performances with measured onset variation, compare pre/post quantization displacement, measure duration variance, and conduct listening tests to assess whether grid choices retain expressive intent. Composition outputs should be evaluated through computational checks and blinded human listening.

9. Limitations and future work

The current normalizer focuses on note-on and note-off events and does not guarantee preservation of all controllers, aftertouch, pitch bend, program changes, SysEx or automation. It intentionally flattens tempo and meter. The scale system is 12-TET and requires the user to select tonic and scale; it does not automatically detect a key. Chord labels are approximations for some scale families. The reference renderer is intentionally simple. The RPP inventory parser is not a full REAPER project interpreter. The test suite is synthetic and no peer-reviewed user study has been completed.

Future work should include a broader MIDI event-preservation mode, optional tempo-map-aware quantization, key/scale hypotheses with uncertainty estimates, user-defined interval recipes, microtonal support, improved voicing and voice-leading constraints, SoundFont-backed rendering, corpus-based evaluation and a machine-readable experiment archive.

References

[1] Mido. *Standard MIDI Files - time attributes, PPQ, tempo and tick conversion.* https://mido.github.io/mido/files/midi.html

[2] Cockos. *REAPER User Guide*, section on REAPER file types (.RPP). https://reaper.fm/userguide/ReaperUserGuide716c.pdf

[3] Cockos. *REAPER ReaScript API documentation.* https://www.reaper.fm/sdk/reascript/reascripthelp

[4] ReaTeam. *REAPER CLI options.* https://github.com/ReaTeam/Doc/blob/master/REAPER-CLI.md

[5] Streamlit. *Streamlit Community Cloud documentation.* https://docs.streamlit.io/deploy/streamlit-community-cloud

[6] FluidSynth. *User Manual.* https://github.com/FluidSynth/fluidsynth/blob/master/doc/wiki/UserManual.md

Publication status. This document is an academic-style technical note accompanying a software prototype. It is not a peer-reviewed journal article. The included unit tests are synthetic and should not be interpreted as evidence of perceptual quality, transcription accuracy or general musical effectiveness.